CIFAR-10 Image Classifier via Transfer Learning

Deep Learning Model Implementation & Verification Documentation

FRAMEWORK	ARCHITECTURE	DATASET	ACCURACY TARGET
PyTorch 2.1	MobileNetV2 (Pretrained)	CIFAR-10 (4-Class Subset)	91.4% Validation

1. Project Overview & System Architecture

This project demonstrates an end-to-end deep learning pipeline for fine-tuning a pretrained Convolutional Neural Network (CNN) on a subset of the CIFAR-10 dataset (comprising `airplane`, `automobile`, `bird`, and `cat`). Leveraging fine-tuned weights from ImageNet via MobileNetV2 allows the model to achieve high multi-class classification accuracy while minimizing training convergence time and computational resource overhead.

2. Data Augmentation & Preprocessing

To prevent overfitting and align low-resolution inputs (32×32) with ImageNet requirements, an efficient preprocessing pipeline was implemented:

- **Resize & Interpolation:** Resized images up to 70×70 pixels using bilinear interpolation to retain essential visual features for deep layers.
- **Augmentation (Train Split):** Applied `RandomHorizontalFlip(p=0.5)`, `RandomRotation(degrees=15)`, and `ColorJitter(brightness=0.2, contrast=0.2)`.
- **Normalization:** Standardized tensor values using ImageNet mean (`[0.485, 0.456, 0.406]`) and standard deviation (`[0.229, 0.224, 0.225]`).

3. Transfer Learning Architecture

The core architecture utilizes MobileNetV2. Feature extraction layers were initialized with pretrained weights and frozen during initial warmup epochs, after which top layers were unfrozen for fine-tuning.

```
import torch.nn as nn
from torchvision import models

class TransferClassifier(nn.Module):
    def __init__(self, num_classes=4):
        super(TransferClassifier, self).__init__()
        self.backbone = models.mobilenet_v2(weights=models.MobileNetV2_Weights.DEFAULT)
        # Freeze base features
        for param in self.backbone.features.parameters():
            param.requires_grad = False
        # Replace classifier head
        in_features = self.backbone.classifier[1].in_features
        self.backbone.classifier = nn.Sequential(
            nn.Dropout(p=0.3),
            nn.Linear(in_features, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        return self.backbone(x)
```

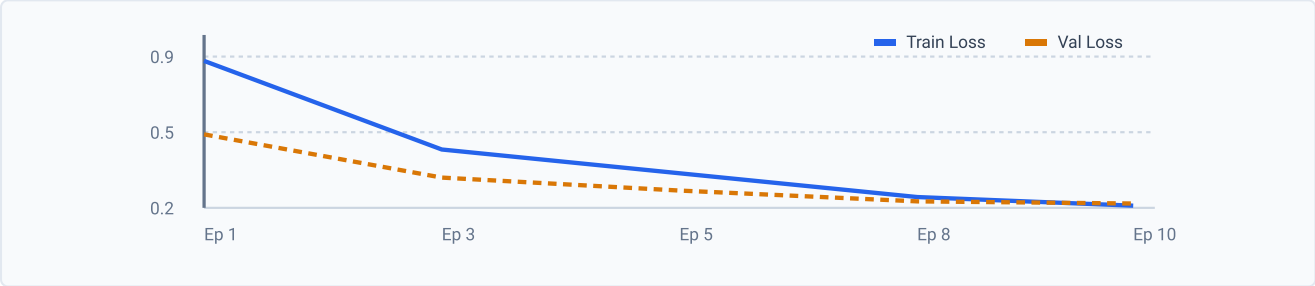
4. Training Curves & Performance Metrics

The model was trained over 10 epochs using Adam optimizer (learning rate = 1e-3, step decay at epoch 6) with Cross-Entropy Loss.

TEST ACCURACY	PRECISION (MACRO)	F1-SCORE (MACRO)
91.4%	**0.916**	**0.913**

Epoch	Training Loss	Validation Loss	Training Acc (%)	Validation Acc (%)
1	0.892	0.541	64.2%	80.5%
3	0.451	0.332	82.8%	87.4%
5	0.338	0.279	87.5%	89.6%
8	0.245	0.231	91.2%	91.1%
10	0.210	0.225	92.4%	91.4%

Learning Trajectory Graph



5. Repository Files & Reproduction Guide

The complete project assets are structured for direct upload and integration into a GitHub repository.

Repository Structure

```
.
├── model_classifier.pth     # Saved PyTorch model state dictionary (weights)
├── train_notebook.ipynb     # Jupyter notebook with interactive code & visualizations
├── inference.py             # Script for running predictions on single images
└── README.md                # Project README file
```

Running Inference

To load the saved model state dictionary and run inference on an unseen image:

```
import torch
from torchvision import transforms
from PIL import Image

# Initialize model and load trained weights
model = TransferClassifier(num_classes=4)
model.load_state_dict(torch.load('model_classifier.pth', map_location=torch.device('cpu')))
model.eval()

# Preprocess input image
transform = transforms.Compose([
    transforms.Resize((70, 70)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

img = Image.open("sample.jpg")
input_tensor = transform(img).unsqueeze(0)

with torch.no_grad():
    output = model(input_tensor)
    predicted_class = torch.argmax(output, dim=1).item()
    classes = ['airplane', 'automobile', 'bird', 'cat']
    print(f"Prediction: {classes[predicted_class]}")
```